ASSIGNMENT-18.5

A.Laxmi Prasanna
2303A51033
Batch-23

Task 1 – IP Address Location Tracker API

Task:
Find geographical location details of an IP address.
Instructions:
• Accept IP input from user.
• Handle invalid IP format and API failures.
• Display city, country, ISP, and region.
Expected Output:
• A script that tracks IP location with proper exception handling.

```python
'''use IP Address Location Tracker API
design geographical location details of an IP address'''
import requests
def get_ip_location(ip_address):
    url = f"http://ip-api.com/json/{ip_address}"
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        if data['status'] == 'success':
            return {
                'ip': ip_address,
                'country': data.get('country', 'N/A'),
                'region': data.get('regionName', 'N/A'),
                'city': data.get('city', 'N/A'),
                'latitude': data.get('lat', 'N/A'),
                'longitude': data.get('lon', 'N/A'),
                'isp': data.get('isp', 'N/A')
            }
        else:
            return {'error': f"Failed to retrieve location for IP: {ip_address}"}
    else:
        return {'error': f"HTTP error {response.status_code} for IP: {ip_address}"}
if __name__ == "__main__":
    ip = input("Enter an IP address: ")
    location_info = get_ip_location(ip)
    print(location_info)
```

OUTPUT:

```
Enter an IP address:
{'ip': '', 'country': 'India', 'region': 'Telangana', 'city': 'Warangal', 'latitude': 17.9958, 'longitude': 79.57
82, 'isp': 'RailTel Corporation Of India Ltd'}
PS C:\Users\ADMIN> 
```

Task 2 – YouTube Channel Statistics API

Task:
Fetch YouTube channel details using Google API.
Instructions:
• Input channel ID or name.
• Handle quota errors, invalid ID, request failures.

• Display subscribers, views, and video count.
Expected Output:
• A working YouTube stats fetcher with error handling.

```python
'''design Fetch YouTube channel details
using YouTube Channel Statistics API and google API '''
import requests
import json
def fetch_channel_details(api_key, channel_id):
    url = f'https://www.googleapis.com/youtube/v3/channels?part=snippet,statistics&id={channel_id}&ke
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()
        if 'items' in data and len(data['items']) > 0:
            channel_info = data['items'][0]
            channel_details = {
                'title': channel_info['snippet']['title'],
                'description': channel_info['snippet']['description'],
                'subscriber_count': channel_info['statistics']['subscriberCount'],
                'video_count': channel_info['statistics']['videoCount'],
                'view_count': channel_info['statistics']['viewCount']
            }
            return channel_details
        else:
            print('Channel not found.')
            return None
    else:
        print(f'Error fetching data: {response.status_code}')
        return None
```

OUTPUT:

```
AC Project/Q218.5.py"
Enter YouTube Channel ID: https://www.youtube.com/@saregamasout
Error fetching data: 400
```

Task 3– Public Transport Live Status API
Task:
Connect to a Transport API to check live bus/train status.
Instructions:
• Input route or train number.
• Handle invalid input, unavailable service, timeout issues.
• Display arrival/departure updates.
Expected Output:
• A real-time transport tracker with retry mechanism.

```python
'''design Public Transport Live Status API
use Transport API to check live bus/train status'''
import requests
API_KEY = "YOUR_API_KEY"  # Replace with your Transport API key
def get_transit_details(source, destination):
    url = "https://maps.googleapis.com/maps/api/directions/json"
    params = {
        "origin": source,
        "destination": destination,
        "mode": "transit",
        "key": API_KEY
    }
    response = requests.get(url, params=params)
    data = response.json()
    if data["status"] == "OK":
        route = data["routes"][0]["legs"][0]
        print("From:", route["start_address"])
        print("To:", route["end_address"])
        print("Duration:", route["duration"]["text"])
        print("\nSteps:")
```

OUTPUT:

```
AC Project/Q318.5.py
Enter source: chenni central
Enter destination: tambaram
Error: REQUEST_DENIED
PS C:\Users\ADMIN>
```

Task 4 – Online Quiz Questions API

Task:

Fetch quiz questions from an Online Trivia API.

Instructions:

- Request 5 multiple-choice questions.
- Handle API errors, invalid category selection.
- Display questions neatly.

Expected Output:

- A quiz generator script with error-safe API handling.

```python
'''use Fetch quiz questions from an Online Trivia API.
design Online Quiz Questions API'''
import requests
import json
def fetch_online_quiz_questions(api_url):
    response = requests.get(api_url)
    if response.status_code == 200:
        data = response.json()
        return data['results']  # Assuming the API returns a 'results' field with quiz questions
    else:
        print(f'Error fetching data: {response.status_code}')
        return None
if __name__ == "__main__":
    api_url = 'https://opentdb.com/api.php?amount=10&type=multiple'  # Example API URL for fetching q
    quiz_questions = fetch_online_quiz_questions(api_url)
    if quiz_questions:
        for idx, question in enumerate(quiz_questions):
            print(f"Question {idx + 1}: {question['question']}")
            print(f"Options: {', '.join(question['incorrect_answers'] + [question['correct_answer']])
            print(f"Correct Answer: {question['correct_answer']}\n")

```

OUTPUT:

```
Question 1: In &quot;Sonic the Hedgehog 3&quot; for the Sega Genesis, what is the color of the second Chaos Emera
ld you can get from Special Stages?
Options: Blue, Green, Magenta, Orange
Correct Answer: Orange

Question 2: In Magic: The Gathering, what card&#039;s flavor text is &quot;Catch!&quot;?
Options: Stone-Throwing Devils, Ember Shot, Throwing Knife, Lava Axe
```

Task 5– E-Commerce Product Price Comparison API

Task:

Integrate a Product Search API to fetch item pricing details.

Instructions:

• User enters product name.

• Handle missing results, invalid API key, slow responses.

• Display product name, price, and availability.

Expected Output:

• A product search tool with graceful failure handling.

```python
'''design Integrate a Product Search API to fetch item pricing details.
use E-Commerce Product Price Comparison API'''
import requests
import json
def fetch_product_price(api_key, product_name):
    url = "https://serpapi.com/search.json"
    params = {
        "engine": "google_shopping",
        "q": product_name,
        "api_key": api_key
    }
    response = requests.get(url, params=params)
    data = response.json()
    if "shopping_results" in data:
        product = data["shopping_results"][0]
        return {
            "product_name": product.get("title"),
            "price": product.get("price"),
            "source": product.get("source")
        }
    else:
        print("No product found")
```

OUTPUT:

```
Enter product name: iphone 15
No product found
```